Exercises week 37

FYS-STK3155/4155, fall 2026 — deadline Friday September 11 at midnight.

Implementing gradient descent for ordinary least squares and Ridge regression

The aim this week is that you write, and *check*, your own gradient descent code for OLS and Ridge regression, and that you understand what the learning rate can and cannot be. Everything here is reused directly in part e) of Project 1, and exercise 6 opens part f). The code developed in the Tuesday session (`week37tuesday.ipynb`, Case 1) is meant to be reused.

After completing these exercises you will have

- your own code for the simplest gradient descent approach applied to ordinary least squares (OLS) and Ridge regression, with a stopping criterion;
- compared the analytical expressions for the OLS and Ridge parameters with the gradient descent results;
- explored the role of the learning rate η in gradient descent and of the hyperparameter λ in Ridge regression, and related the largest usable learning rate to the largest eigenvalue of the Hessian (Section 4.5 of the lecture notes);
- checked your analytical gradient against automatic differentiation (Section 4.14) and added momentum (Section 4.6);
- scaled the data properly.

Reading: Chapter 4, Sections 4.1–4.6 and 4.14 of the book; Goodfellow et al., Chapter 4 and Sections 8.1–8.3.

A simple one-dimensional second-order polynomial

We start with a very simple function,

$$f(x) = 2 - x + 5x^2,$$

defined for $x \in [-2, 2]$. You can add noise if you wish (the Tuesday session used Gaussian noise with standard deviation 0.5). We fit this function with a polynomial ansatz. The easiest thing is to set up a second-order polynomial and see if you can recover the above function; feel free to play around with higher-order polynomials — the Tuesday session shows that a degree-5 fit of the same data is a much harder problem for gradient descent, and exercise 4 asks you why.

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(2026)
n = 100
x = rng.uniform(-2.0, 2.0, n)
y = 2.0 - x + 5.0 * x**2 + 0.5 * rng.standard_normal(n)   # drop the noise t
```

Exercise 1: scale your data

Before fitting a regression model it is good practice to standardise the features. This ensures all features are on a comparable scale, which is especially important when using regularisation — and, as Section 4.5 of the lecture notes shows, it is also what makes gradient descent converge in a reasonable number of iterations. Here we standardise, scaling each feature to have mean 0 and standard deviation 1.

1a)

Set up the design matrix $\boldsymbol{X}$ with the columns x and x^2 (no column of ones). Compute the mean and standard deviation of each column, subtract the mean and divide by the standard deviation.

We also centre the target $\boldsymbol{y}$ to mean 0. Centring $\boldsymbol{y}$ and each feature means the model does not need a separate intercept term: the data are shifted so that the intercept is effectively 0. (In practice one can include an intercept in the model and leave it unpenalised, see Section 3.13 of the lecture notes; here we simplify by centring.)

In [3]:
```
def standardize(X, y):
    """
    This function standarized features so they have zero mean and
    unit variance for each feature.
    """
    X_mean = X.mean(axis=0)
    X_std = X.std(axis=0)
    X_std[X_std == 0] = 1  # safeguard to avoid division by zero for constar
    X_norm = (X - X_mean) / X_std

    # Center the target to zero mean (optional, to simplify intercept handli
    y_mean = np.mean(y)
    y_centered = y - y_mean

    return X_norm, y_centered


X = np.column_stack([x, x**2])
n_features = X.shape[1]
X_norm, y_centered = standardize(X, y)
```

Fill in the necessary details. Do we need to centre the y-values? What would go wrong, for OLS and for Ridge, if we did not?

After this preprocessing each column of $\boldsymbol{X}_{\text{norm}}$ has mean zero and standard deviation 1 and $\boldsymbol{y}_{\text{centered}}$ has mean 0. This makes the optimisation landscape nicer and ensures that the penalty $\lambda \sum_j \theta_j^2$ in Ridge regression treats each coefficient fairly, since the features are on the same scale.

Solution Exercise 1a)

We need to centre the target values $\boldsymbol{y}$ because the design matrix does not contain a column of ones. That is, the model has no explicit intercept term. After centering and standardising the columns of $\boldsymbol{X}$, each feature (column) has mean zero. The model predictions are found from

$$\hat{\boldsymbol{y}}_{\text{centered}} = \boldsymbol{X}_{\text{norm}}\hat{\boldsymbol{\theta}},$$

which will also then have zero mean. If we did not center $\boldsymbol{y}$, the model would not be able to reproduce the mean value of $\boldsymbol{y}$ because the intercept is missing. The intercept is what allows the model to have a non-zero mean when the features are centred. Since the predictions now have zero mean, we need to recover the original scale by adding back the mean of $\boldsymbol{y}$, so $\hat{\boldsymbol{y}}_{\text{original scale}} = \bar{\boldsymbol{y}} + \boldsymbol{X}_{\text{norm}}\hat{\boldsymbol{\theta}}$.

Furthermore, the fitted OLS coefficients are not affected by this centering of $\boldsymbol{y}$. This can be seen by writing

$$\boldsymbol{y} = \boldsymbol{y}_{\text{centered}} + \bar{y}\mathbf{1}.$$

Multiplying both sides with $\boldsymbol{X}_{\text{norm}}^T$ gives

$$\boldsymbol{X}_{\text{norm}}^T\boldsymbol{y} = \boldsymbol{X}_{\text{norm}}^T\boldsymbol{y}_{\text{centered}} + \bar{y}\boldsymbol{X}_{\text{norm}}^T\mathbf{1}.$$

The columns of $\boldsymbol{X}_{\text{norm}}$ have zero mean, so $\boldsymbol{X}_{\text{norm}}^T\mathbf{1} = 0$ and

$$\boldsymbol{X}_{\text{norm}}^T\boldsymbol{y} = \boldsymbol{X}_{\text{norm}}^T\boldsymbol{y}_{\text{centered}}.$$

The OLS solution is

$$\hat{\boldsymbol{\theta}} = (\boldsymbol{X}_{\text{norm}}^T\boldsymbol{X}_{\text{norm}})^{-1}\boldsymbol{X}_{\text{norm}}^T\boldsymbol{y}.$$

Since $\boldsymbol{X}_{\text{norm}}^T\boldsymbol{y} = \boldsymbol{X}_{\text{norm}}^T\boldsymbol{y}_{\text{centered}}$, then

$$\hat{\boldsymbol{\theta}} = (\boldsymbol{X}_{\text{norm}}^T\boldsymbol{X}_{\text{norm}})^{-1}\boldsymbol{X}_{\text{norm}}^T\boldsymbol{y}_{\text{centered}}.$$

Thus centring $\boldsymbol{y}$ does not change the OLS coefficients. The same argument applies to Ridge regression, since the additional regularisation term does not depend on $\boldsymbol{y}$.

1b)

The true function has coefficients $(-1, 5)$ on (x, x^2). Which coefficients do you expect to find on the *standardised* columns, and how do you convert the fitted parameters back to the original scale?

Solution exercise 1b)

The original function $y = 2 - x + 5x^2$ has coefficients $\theta_1 = -1$ and $\theta_2 = 5$ for the features $x_1 = x$ and $x_2 = x^2$. The standardised columns are

$$x_1^{\text{std}} = \frac{x_1 - \bar{x_1}}{\sigma_1}, \qquad x_2^{\text{std}} = \frac{x_2 - \bar{x_2}}{\sigma_2},$$

where $\sigma_1 = \text{std}(x)$ and $\sigma_2 = \text{std}(x^2)$. To find the coefficients for x_1^{std} and x_2^{std} we solve for x_1 and x_2, respectively. Then

$$x_1 = \bar{x_1} + \sigma_1 x_1^{\text{std}}, \qquad x_2 = \bar{x_2} + \sigma_2 x_2^{\text{std}}.$$

Putting these into the original function y gives

$$\begin{aligned} y &= 2 - (\bar{x_1} + \sigma_1 x_1^{\text{std}}) + 5(\bar{x_2} + \sigma_2 x_2^{\text{std}}) && (1) \\ &= (2 - \bar{x_1} + 5\bar{x_2}) - \sigma_1 x_1^{\text{std}} + 5\sigma_2 x_2^{\text{std}}, && (2) \end{aligned}$$

where the first parenthesis is the intercept. The coefficients are therefore

$$\theta_1^{\text{std}} = -\sigma_1 \quad , \quad \theta_2^{\text{std}} = 5\sigma_2.$$

To convert back, we substitute $x = \bar{x} + \sigma_1 x^{\text{std}}$ in θx:

$$\theta x = \theta(\bar{x} + \sigma_1 x^{\text{std}}) = \theta\bar{x} + \theta\sigma_1 x^{\text{std}},$$

so the coefficient in front of x_1^{std} is $\theta^{\text{std}} = \theta\sigma_1$. That is,

$$\theta_1 = \frac{\theta_1^{\text{std}}}{\sigma_1} \quad , \quad \theta_2 = \frac{\theta_2^{\text{std}}}{\sigma_2}.$$

Exercise 2: calculate the gradients and the Hessian

Find the gradients of the OLS and Ridge cost functions,

$$C_{\text{OLS}}(\boldsymbol{\theta}) = \frac{1}{n}\|\boldsymbol{X\theta} - \boldsymbol{y}\|_2^2, \qquad C_{\text{Ridge}}(\boldsymbol{\theta}) = \frac{1}{n}\|\boldsymbol{X\theta} - \boldsymbol{y}\|_2^2 + \lambda\boldsymbol{\theta}^T\boldsymbol{\theta},$$

with respect to $\boldsymbol{\theta}$ (Eqs. (4.13) and (4.17) of the lecture notes). Find also the Hessian matrices, and show that the OLS Hessian is positive semi-definite and the Ridge Hessian

positive definite for $\lambda > 0$ — that is, that both cost functions are convex (Section 4.1). Why does this matter for gradient descent?

Setting the Ridge gradient to zero, show that the minimiser is $\hat{\boldsymbol{\theta}} = (\boldsymbol{X}^T\boldsymbol{X} + n\lambda\boldsymbol{I})^{-1}\boldsymbol{X}^T\boldsymbol{y}$. Where does the factor n come from, and what is the corresponding `alpha` of `scikit-learn`'s `Ridge`? (Section 3.10 and Eq. (3.95).)

Solution exercise 2

We start with the OLS cost function

$$C_{\mathrm{OLS}}(\boldsymbol{\theta}) = \frac{1}{n}\|X\boldsymbol{\theta} - \mathbf{y}\|_2^2.$$

Using

$$\|X\boldsymbol{\theta} - \mathbf{y}\|_2^2 = (X\boldsymbol{\theta} - \mathbf{y})^T(X\boldsymbol{\theta} - \mathbf{y}),$$

we can write

$$C_{\mathrm{OLS}}(\boldsymbol{\theta}) = \frac{1}{n}\left[\boldsymbol{\theta}^T X^T X\boldsymbol{\theta} - \boldsymbol{\theta}^T X^T\mathbf{y} - \mathbf{y}^T X\boldsymbol{\theta} + \mathbf{y}^T\mathbf{y}\right].$$

Consider a design matrix with an intercept and one feature,

$$\boldsymbol{\theta} = \begin{pmatrix}\theta_0\\ \theta_1\end{pmatrix}, \qquad X = \begin{pmatrix}1 & x_0\\ \vdots & \vdots\\ 1 & x_{n-1}\end{pmatrix}.$$

Then

$$C_{\mathrm{OLS}}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=0}^{n-1}(\theta_0 + \theta_1 x_i - y_i)^2.$$

Taking the derivative with respect to θ_0,

$$\frac{\partial C}{\partial \theta_0} = \frac{2}{n}\sum_{i=0}^{n-1}(\theta_0 + \theta_1 x_i - y_i).$$

Similarly,

$$\frac{\partial C}{\partial \theta_1} = \frac{2}{n}\sum_{i=0}^{n-1}x_i(\theta_0 + \theta_1 x_i - y_i).$$

Therefore,

$$\nabla_{\boldsymbol{\theta}} C_{\mathrm{OLS}} = \frac{2}{n}\begin{pmatrix} \sum_{i=0}^{n-1}(\theta_0+\theta_1 x_i - y_i) \\ \sum_{i=0}^{n-1} x_i(\theta_0+\theta_1 x_i - y_i) \end{pmatrix}.$$

This can be written compactly in matrix form as

$$\boxed{\nabla_{\boldsymbol{\theta}} C_{\mathrm{OLS}} = \frac{2}{n}X^T(X\boldsymbol{\theta}-\mathbf{y})}.$$

OLS Hessian

The Hessian is the matrix of second derivatives,

$$H_{\mathrm{OLS}} = \nabla^2_{\boldsymbol{\theta}} C_{\mathrm{OLS}}.$$

For the two-parameter example,

$$H_{\mathrm{OLS}} = \frac{2}{n}\begin{pmatrix} n & \sum_{i=0}^{n-1} x_i \\ \sum_{i=0}^{n-1} x_i & \sum_{i=0}^{n-1} x_i^2 \end{pmatrix}.$$

Since

$$X^T X = \begin{pmatrix} n & \sum_i x_i \\ \sum_i x_i & \sum_i x_i^2 \end{pmatrix},$$

we have

$$\boxed{H_{\mathrm{OLS}} = \frac{2}{n}X^T X}.$$

To determine whether this matrix is positive semidefinite, consider an arbitrary vector $\mathbf{z}$. Then

$$\mathbf{z}^T H_{\mathrm{OLS}}\mathbf{z} = \frac{2}{n}\mathbf{z}^T X^T X\mathbf{z} = \frac{2}{n}(X\mathbf{z})^T(X\mathbf{z}) = \frac{2}{n}\|X\mathbf{z}\|_2^2 \geq 0.$$

Therefore,

$$H_{\mathrm{OLS}} \geq 0,$$

so the OLS cost function is convex.

Ridge regression

The Ridge cost function is

$$C_{\text{Ridge}}(\boldsymbol{\theta}) = \frac{1}{n}\|X\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \boldsymbol{\theta}^T\boldsymbol{\theta}.$$

Since

$$\boldsymbol{\theta}^T\boldsymbol{\theta} = \sum_j \theta_j^2,$$

the derivative of the regularization term is

$$\nabla_{\boldsymbol{\theta}} \left(\lambda \boldsymbol{\theta}^T\boldsymbol{\theta}\right) = 2\lambda\boldsymbol{\theta}.$$

Therefore the Ridge gradient is simply the OLS gradient plus the regularization contribution,

$$\boxed{\nabla_{\boldsymbol{\theta}} C_{\text{Ridge}} = \frac{2}{n}X^T(X\boldsymbol{\theta} - \mathbf{y}) + 2\lambda\boldsymbol{\theta}}.$$

Ridge Hessian

Differentiating the gradient once more gives

$$H_{\text{Ridge}} = H_{\text{OLS}} + 2\lambda I.$$

Thus,

$$\boxed{H_{\text{Ridge}} = \frac{2}{n}X^TX + 2\lambda I}.$$

For an arbitrary nonzero vector $\mathbf{z}$,

$$\begin{aligned}\mathbf{z}^T H_{\text{Ridge}}\mathbf{z} &= \frac{2}{n}\mathbf{z}^T X^T X\mathbf{z} + 2\lambda \mathbf{z}^T\mathbf{z} \\ &= \frac{2}{n}\|X\mathbf{z}\|_2^2 + 2\lambda\|\mathbf{z}\|_2^2.\end{aligned}$$

For $\lambda > 0$ and $\mathbf{z} \neq 0$,

$$\frac{2}{n}\|X\mathbf{z}\|_2^2 + 2\lambda\|\mathbf{z}\|_2^2 > 0.$$

Therefore,

$$H_{\text{Ridge}} > 0$$

for $\lambda > 0$, and the Ridge cost function is strictly convex.

Why does convexity matter for gradient descent?

For a convex cost function, any local minimum is also a global minimum. Therefore, gradient descent cannot become trapped in a non-global local minimum.

For Ridge regression with $\lambda > 0$, the cost function is strictly convex, so there is a unique global minimum.

Analytical Ridge solution

To find the minimum, we set the Ridge gradient equal to zero:

$$\frac{2}{n}X^T(X\hat{\boldsymbol{\theta}} - \mathbf{y}) + 2\lambda\hat{\boldsymbol{\theta}} = 0.$$

Dividing by 2,

$$\frac{1}{n}X^T(X\hat{\boldsymbol{\theta}} - \mathbf{y}) + \lambda\hat{\boldsymbol{\theta}} = 0.$$

Multiplying by n gives

$$X^TX\hat{\boldsymbol{\theta}} - X^T\mathbf{y} + n\lambda\hat{\boldsymbol{\theta}} = 0.$$

Hence,

$$(X^TX + n\lambda I)\hat{\boldsymbol{\theta}} = X^T\mathbf{y},$$

and therefore

$$\boxed{\hat{\boldsymbol{\theta}} = (X^TX + n\lambda I)^{-1}X^T\mathbf{y}}.$$

The factor n appears because the data-fitting term in our cost function contains a factor $1/n$, while the regularization term does not.

Relation to scikit-learn's `Ridge`

Scikit-learn defines the Ridge objective as

$$\|X\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \alpha\|\boldsymbol{\theta}\|_2^2.$$

Our objective is

$$\frac{1}{n}\|X\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda\|\boldsymbol{\theta}\|_2^2.$$

Multiplying our objective by n does not change its minimum:

$$\|X\boldsymbol{\theta} - \mathbf{y}\|_2^2 + n\lambda\|\boldsymbol{\theta}\|_2^2.$$

Thus the corresponding scikit-learn parameter is

$$\boxed{\alpha = n\lambda}.$$

Exercise 3: the analytical formulae for OLS and Ridge regression

Solution 3a)

In [4]:
```
from sklearn.linear_model import Ridge


def closed_form_thetas(X, y, lmbda):
    """
    Computes closed-form thetas for both OLS and Ridge.
    X and y must is assumed scaled.
    """
    # Analytical forms: theta_Ridge = (X^T X + n*lambda*I)^{-1} X^T y and th
    n_features = X.shape[1]
    I = np.eye(n_features)
    n = len(y)

    X_TX = X.T @ X
    X_Ty = X.T @ y
    theta_OLS = np.linalg.solve(X_TX, X_Ty)
    theta_Ridge = np.linalg.solve(X_TX + n * lmbda * I, X_Ty)

    return theta_OLS, theta_Ridge


# compute closed-form thetas for OLS and Rigde
lmbda = 0.1
theta_closed_formOLS, theta_closed_formRidge = closed_form_thetas(X_norm, y_

# compare with scikit-learn
theta_sklearnRidge = Ridge(alpha=n*lmbda, fit_intercept=False).fit(X_norm, y

print("Closed-form OLS coefficients:             ", theta_closed_formOLS)
print(f"Closed-form Ridge coefficients (lam={lmbda}): ", theta_closed_formRi
print(f"Scikit-learn Ridge coefficients (lam={lmbda}):", theta_sklearnRidge)
```

```
Closed-form OLS coefficients:              [-1.0773505  5.854734 ]
Closed-form Ridge coefficients (lam=0.1):  [-0.92026706  5.30556833]
Scikit-learn Ridge coefficients (lam=0.1): [-0.92026706  5.30556833]
```

Comments 3a):

The analytical Ridge coefficients match the Scikit-learn coefficients (using the same $\lambda = 0.1$ for both Ridge methods).

3b)

Explore the results as functions of different values of the hyperparameter λ. See for example exercise 4 from week 36, and Section 3.14 of the lecture notes for how λ should be chosen in practice.

Solution 3b)

In [5]:
```
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error


def ridge_kfold(X, y, lmbda, k=5):
    """
    Performs k-fold cross validation (k=5 folds default) for a fixed lambda
    X and y are assumed scaled.
    """
    kf = KFold(n_splits=k, shuffle=True, random_state=2026)
    mse_list = []

    for train_idx, test_idx in kf.split(X):
        X_tr = X[train_idx]
        X_te = X[test_idx]

        y_tr = y[train_idx]
        y_te = y[test_idx]

        # scale data
        X_mean = np.mean(X_tr, axis=0)
        X_std = np.std(X_tr, axis=0)

        y_mean = np.mean(y_tr)
        y_tr_centered = y_tr - y_mean

        X_tr_norm = (X_tr - X_mean) / X_std
        X_te_norm = (X_te - X_mean) / X_std

        # solve using Ridge
        X_TX = X_tr_norm.T @ X_tr_norm
        X_Ty = X_tr_norm.T @ y_tr_centered

        n = len(y_tr)
        n_features = X_tr_norm.shape[1]
        I = np.eye(n_features)

        theta = np.linalg.solve(X_TX + n*lmbda*I, X_Ty)
        y_pred = X_te_norm @ theta + y_mean

        mse_list.append(mean_squared_error(y_te, y_pred))

    return np.mean(mse_list), np.std(mse_list)


# find best lambda value using CV
lmbdas = np.logspace(-5, 2, 150)
```

```
theta_ridge_vals = np.zeros((n_features, len(lmbdas)))
mean_mse = np.zeros(len(lmbdas))
std_mse = np.zeros(len(lmbdas))

for i, lmbda in enumerate(lmbdas):
    mean_mse[i], std_mse[i] = ridge_kfold(X_norm, y_centered, lmbda, k=5)
    theta_ridge_vals[:, i] = closed_form_thetas(X_norm, y_centered, lmbda)[1

best_idx = np.argmin(mean_mse)
best_lmbda = lmbdas[best_idx]

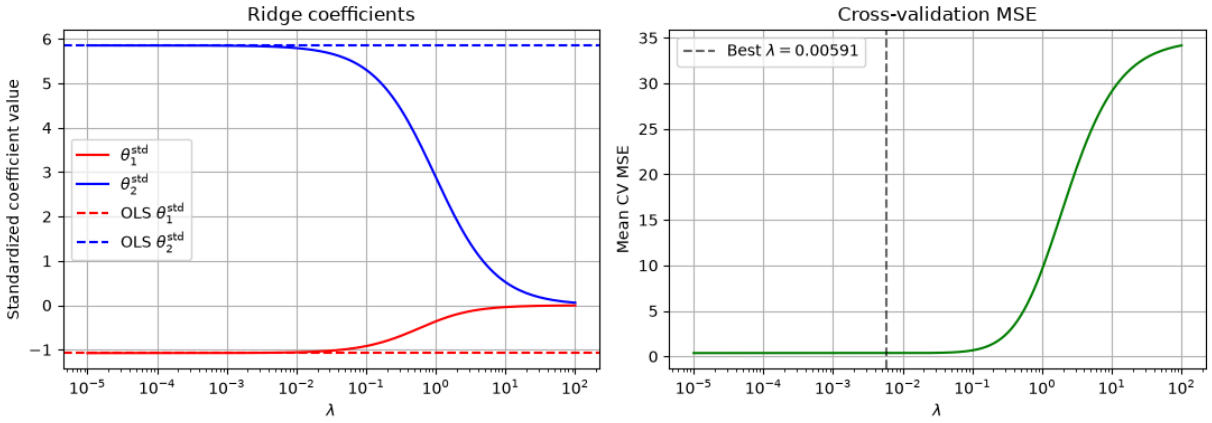
print(f"Best lambda: {best_lmbda:.4f}")
print(f"Minimum CV MSE: {mean_mse[best_idx]:.4f}")
print(f"Standardized coefficients:  theta1 = {-1 * np.std(x):.4f}, theta2 =


# plot Ridge coefficients
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].semilogx(lmbdas, theta_ridge_vals[0, :], color="red", label=r"$\theta_
ax[0].semilogx(lmbdas, theta_ridge_vals[1, :], color="blue", label=r"$\theta
ax[0].axhline(theta_closed_formOLS[0], color="red", linestyle="--", label=r"
ax[0].axhline(theta_closed_formOLS[1], color="blue", linestyle="--", label=r
ax[0].set_xlabel(r"$\lambda$")
ax[0].set_ylabel("Standardized coefficient value")
ax[0].set_title("Ridge coefficients")
ax[0].legend()
ax[0].grid()

# plot cross-validation MSE
ax[1].semilogx(lmbdas, mean_mse, color="green")
ax[1].axvline(best_lmbda, linestyle="--", alpha=0.6, color="black", label=fr
ax[1].set_xlabel(r"$\lambda$")
ax[1].set_ylabel("Mean CV MSE")
ax[1].set_title("Cross-validation MSE")
ax[1].legend()
ax[1].grid()

plt.tight_layout()
plt.show()
```

```
Best lambda: 0.0059
Minimum CV MSE: 0.3642
Standardized coefficients:  theta1 = -1.0387, theta2 = 5.8479
```



Comments 3b):

We explored many different values of the regularization parameter λ and found the best one using k-fold cross validation. The value that minimizes the mean validation MSE was $\lambda \approx 0.0059$. The plot to the left shows the Ridge coefficients for different values of λ, and we see that the coefficients are driven towards zero as the regularization increases, as expected. For the original model, $y = 2 - x + 5x^2$ the coefficients for the standardized model are $\theta_1 = -\sigma_x \approx -1$ and $\theta_2 = 5\sigma_{x^2} \approx 5.8$ (from exercise 1), which agrees with the leftmost plot.

Exercise 4: implementing the simplest form of gradient descent

Alternatively we can fit the model by gradient descent, Eq. (4.15) of the lecture notes,

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}_k).$$

This is useful to visualise the iterative convergence, and it is necessary when n and p are so large that the closed form is too slow or too memory-hungry — or when, as for the Lasso, logistic regression and neural networks, no closed form exists. Use the gradients of exercise 2 and set up, using the template below, your own gradient descent code for OLS and Ridge regression.

4a)

Write first a gradient descent code for OLS only, using the above template. Discuss the results as functions of the learning rate and the number of iterations: for $\eta \in \{0.01, 0.1, 0.5, 0.8\}$, how many iterations are needed to reach the closed-form solution of exercise 3 to a given accuracy, say $\|\boldsymbol{\theta}_k - \hat{\boldsymbol{\theta}}\|_2 < 10^{-6}$? Plot this distance against the iteration number on a logarithmic scale.

4b)

Write then a similar code for Ridge regression. Add a stopping criterion based on the number of iterations *and* on the size of the gradient (or on the difference between the new and old $\boldsymbol{\theta}$). How would you define a stopping criterion, and what goes wrong with a fixed number of iterations alone?

Solution 4a) and b)

In [6]:
```
def gradient_descent(X, y, theta_exact, eta,  method="OLS", lmbda=0.0, num_i
    """
    Performs gradient descent using num_iters number of iterations using OLS
    Stops when the gradient converges for a chosen tolerance.
    (theta, iterations, distances) is returned.
    """
    n = len(y)
    theta = np.zeros(X.shape[1])   # starting guess is zero

    distances = []

    for i in range(num_iters):
        if method == "OLS":
            gradient = (2/n) * X.T @ (X @ theta - y)

        elif method == "Ridge":
            gradient = (2/n) * X.T @ (X @ theta - y) + 2 * lmbda * theta

        else:
            raise RuntimeError("Please choose regression method: OLS or Ridg

        # update theta parameters by moving opposite of gradient
        theta -= eta * gradient

        # compare to closed-form solution
        distance = np.linalg.norm(theta - theta_exact)
        distances.append(distance)

        # stopping criterion if gradient converges
        if np.linalg.norm(gradient) < tol:
            break

    return theta, i + 1, np.array(distances)
```

In [7]:
```
# ---------------------------------
#            Exercise 4a)
# ---------------------------------

etas = [0.01, 0.1, 0.5, 0.8]

plt.figure(figsize=(7, 5))
print(f"Convergence to closed-form theta with 1e-6 accuracy:")

for eta in etas:
    theta_OLS, iter_OLS, distances_OLS = gradient_descent(X_norm, y_centered
```

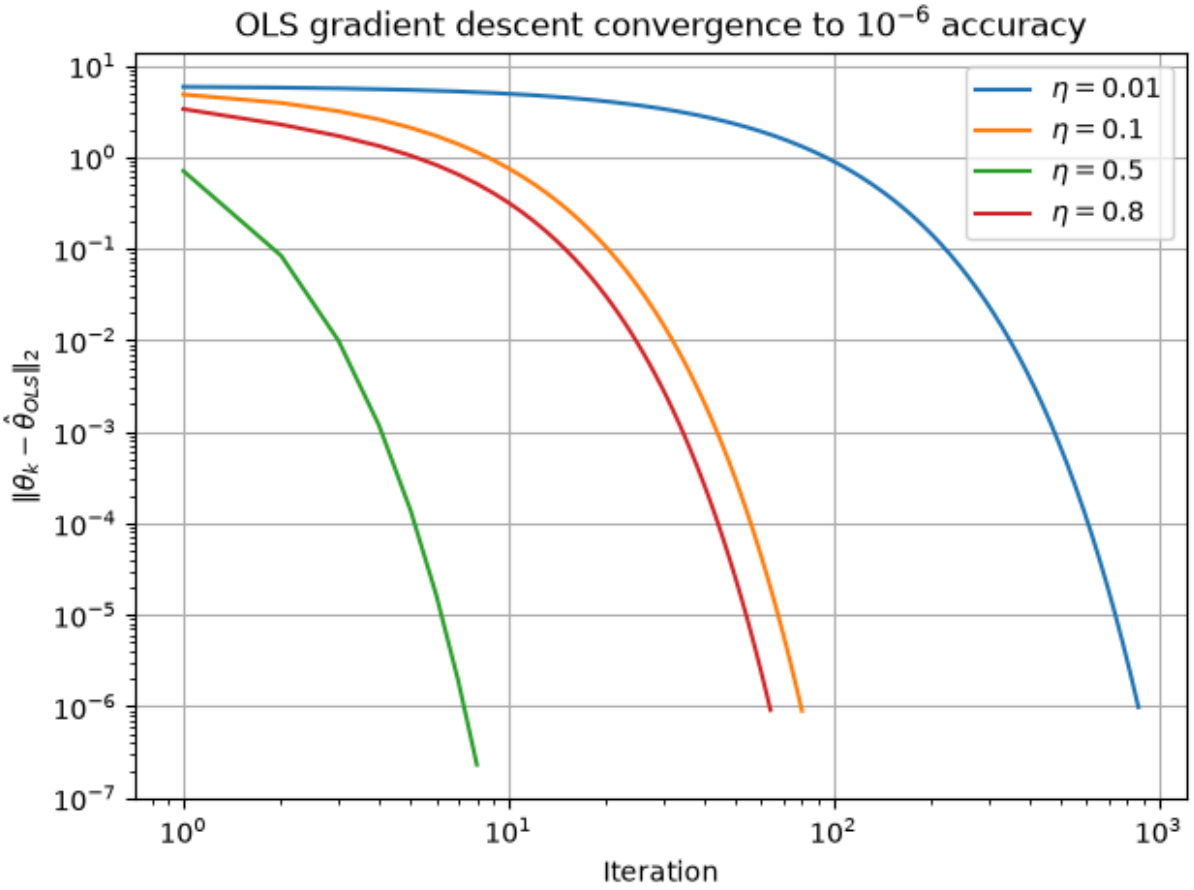
```
    # find iterations needed to reach 1e-6 accuracy
    indices = np.where(distances_OLS < 1e-6)[0]

    if len(indices) > 0:
        iterations= indices[0] + 1
        print(
            f"-----------------------------------------\n"
            f"eta = {eta}:\n"
            f"iterations = {iterations}\n"
            f"theta = {theta_OLS[:iterations]}"
        )
        plt.loglog(range(1, iterations+1), distances_OLS[:iterations], label

    else:
        print(f"eta = {eta}: did not reach 1e-6 accuracy.")

plt.xlabel("Iteration")
plt.ylabel(r"$\|\theta_k-\hat{\theta}_{OLS}\|_2$")
plt.title(r"OLS gradient descent convergence to $10^{-6}$ accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Convergence to closed-form theta with 1e-6 accuracy:
-----------------------------------------
eta = 0.01:
iterations = 867
theta = [-1.07735011  5.8547336 ]
-----------------------------------------
eta = 0.1:
iterations = 80
theta = [-1.07735021  5.85473371]
-----------------------------------------
eta = 0.5:
iterations = 8
theta = [-1.07735048  5.854734  ]
-----------------------------------------
eta = 0.8:
iterations = 64
theta = [-1.07735031  5.8547342 ]
```



Comments 4a):

We see that the learning rate affects the number of iterations needed to reach 10^{-6} accuracy remarkably. For the smallest $\eta = 0.01$, the convergence is slow, needing 867 iterations. For $\eta = 0.5$ the accyracy is reached with just 8 iterations. Increasing to $\eta = 0.8$ makes the convergence slower again, needing 64 iterations, indicating that the learning rate is too large and the updates of the paramaters overshoots the minimum.

In [8]:
```
# ---------------------------------
#            Exercise 4b)
# ---------------------------------

plt.figure(figsize=(7, 5))
print(f"Convergence for stopping criterion tol=1e-8:")

for eta in etas:
    theta_closed_formRidge_best = closed_form_thetas(X_norm, y_centered, lmb
    theta_Ridge, iter_Ridge, distances_Ridge = gradient_descent(X_norm, y_ce

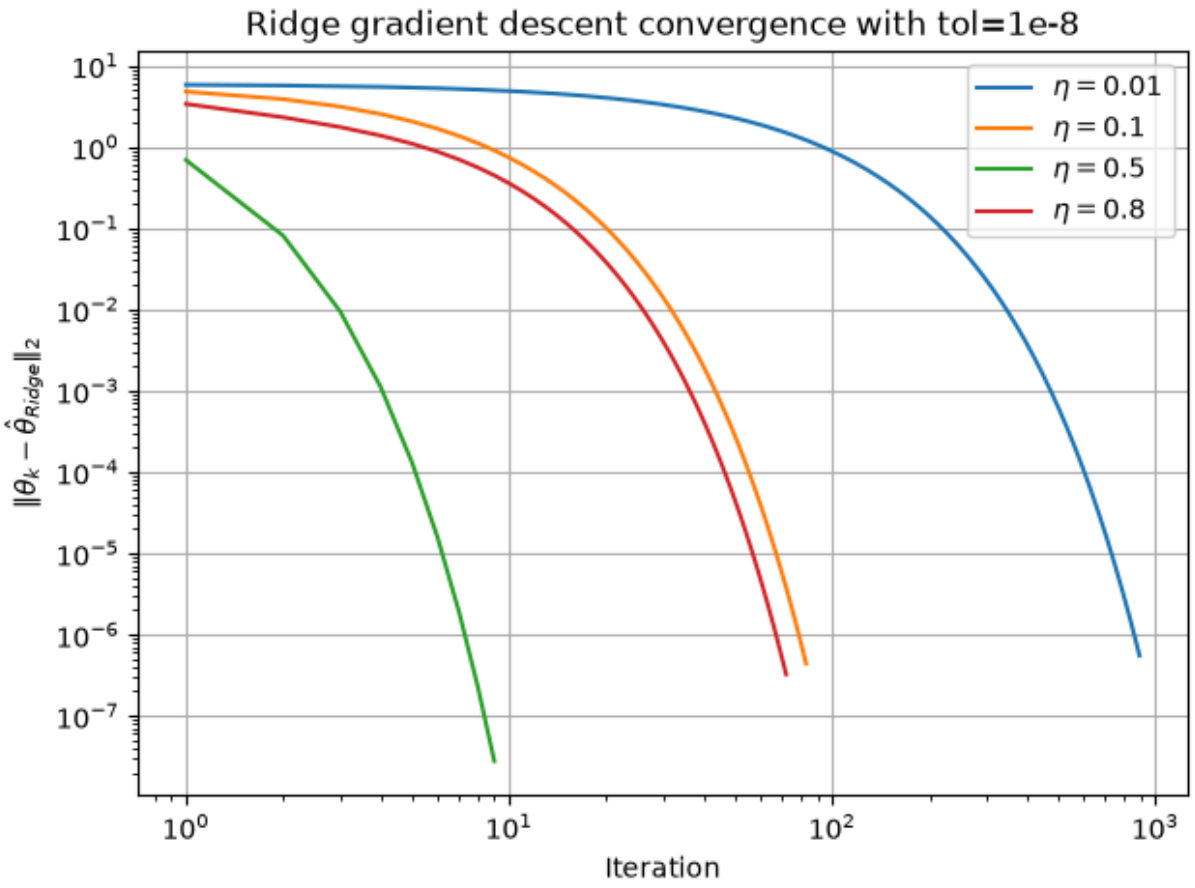
    plt.loglog(range(1, iter_Ridge+1), distances_Ridge, label=fr"$\eta={eta}

    print(f"-----------------------------------------\n"
          f"eta = {eta}:\n"
          f"Gradient descent Ridge: {theta_Ridge}\n"
          f"Closed-form Ridge:      {theta_closed_formRidge_best}\n"
          f"Iterations: {iter_Ridge}"
```

```
    )

plt.xlabel("Iteration")
plt.ylabel(r"$\|\theta_k-\hat{\theta}_{Ridge}\|_2$")
plt.title(r"Ridge gradient descent convergence with tol=1e-8")
plt.legend()
plt.grid()
plt.show()
```

```
Convergence for stopping criterion tol=1e-8:
--------------------------------------------
eta = 0.01:
Gradient descent Ridge: [-1.06681796  5.81907738]
Closed-form Ridge:      [-1.06681834  5.81907777]
Iterations: 893
--------------------------------------------
eta = 0.1:
Gradient descent Ridge: [-1.06681803  5.81907746]
Closed-form Ridge:      [-1.06681834  5.81907777]
Iterations: 83
--------------------------------------------
eta = 0.5:
Gradient descent Ridge: [-1.06681832  5.81907778]
Closed-form Ridge:      [-1.06681834  5.81907777]
Iterations: 9
--------------------------------------------
eta = 0.8:
Gradient descent Ridge: [-1.06681857  5.81907754]
Closed-form Ridge:      [-1.06681834  5.81907777]
Iterations: 72
```



Comments 4b):

For this task we use gradient descent with Ridge regression and use both a maximum amount of iterations and a stopping criterion (`tol`). That way, the algorithm stops both when the max iterations is achieved, as the gradient descent is performed inside a for loop. It stops early if the stopping criterion is reached. We made a generalised function above, `gradient\_descent`, which implements both so this was also used in exercise 4a). But now we stop the loop when `tol=1e-8` is reached withing a maximum of 1000 iterations. This is better than using only a fixed number of iterations, because the number of iterations needed for convergence depends on the chosen learning rate η. Imposing both criterions makes sure that the desired tolerance is reached rather than just running out of iterations.

Here we have tested different values of η, as this affects the convergence. As seen, $\eta = 0.5$ is ideal in this case, and convergence is achieved within 9 iterations. Increasing η further increases the number of iterations needed. Overall, we also see that the gradient descent coefficients agree with the closed form coefficients, as seen from the results printed above when running the code.

4c) The learning rate and the eigenvalues of the Hessian

Compute the eigenvalues $\lambda_{\max}$ and $\lambda_{\min}$ of the Hessian matrix of exercise 2 (use `np.linalg.eigvalsh`). Section 4.5 of the lecture notes shows that gradient descent

converges if and only if $\eta < 2/\lambda_{\max}$, Eq. (4.20), that the fastest convergence is obtained for $\eta^* = 2/(\lambda_{\max} + \lambda_{\min})$, and that the number of iterations grows with the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$, Eq. (4.21). Verify the bound numerically: run your code at $0.99 \cdot 2/\lambda_{\max}$ and at $1.02 \cdot 2/\lambda_{\max}$.

Repeat for a design matrix with the columns $x, x^2, \ldots, x^5$ (standardised). What happens to κ, to $\eta_{\max}$ and to the number of iterations, and why? Which of the two data sets does Ridge regression help more, and in what sense?

Solution 4c)

In [9]:
```
def compute_etas(H):
    """
    Computes max learning rate (eta), the optimal learning rate and
    the condition number (kappa) from the eigenvalues of the Hessian (H).
    """
    eigenvals = np.linalg.eigvalsh(H)
    lmbda_min = eigenvals[0]
    lmbda_max = eigenvals[-1]

    eta_optimal = 2 / (lmbda_max + lmbda_min)
    eta_max = 2 / lmbda_max
    kappa = lmbda_max / lmbda_min

    return eta_optimal, eta_max, kappa
```

In [10]:
```
# ------------------------------
# degree-2 design matrix case:
# ------------------------------

H2 = (2 / len(y_centered)) * X_norm.T @ X_norm

eta_optimal, eta_max, kappa = compute_etas(H2)
theta2, iter2, dist2 = gradient_descent(X_norm, y_centered, theta_closed_for

# testing the boundaries of eta with OLS
eta_max_under = 0.99 * eta_max
eta_max_over = 1.02 * eta_max

theta_under, iter_under, dist_under = gradient_descent(X_norm, y_centered, t
theta_over, iter_over, dist_over = gradient_descent(X_norm, y_centered, thet

# print and plot results
print("Degree 2:")
print("----------------------------")
print(f"eta_max = {eta_max:.4f}")
print(f"eta_optimal = {eta_optimal:.4f}")
print(f"condition number = {kappa:.4f}")
print(f"iterations: {iter2}\n")
print(f"eta below limit (0.99eta_max): {eta_max_under:.4f}")
print(f"final distance: {dist_under[-1]:.4e}\n")
print(f"eta above limit (1.02eta_max): {eta_max_over:.4f}")
```

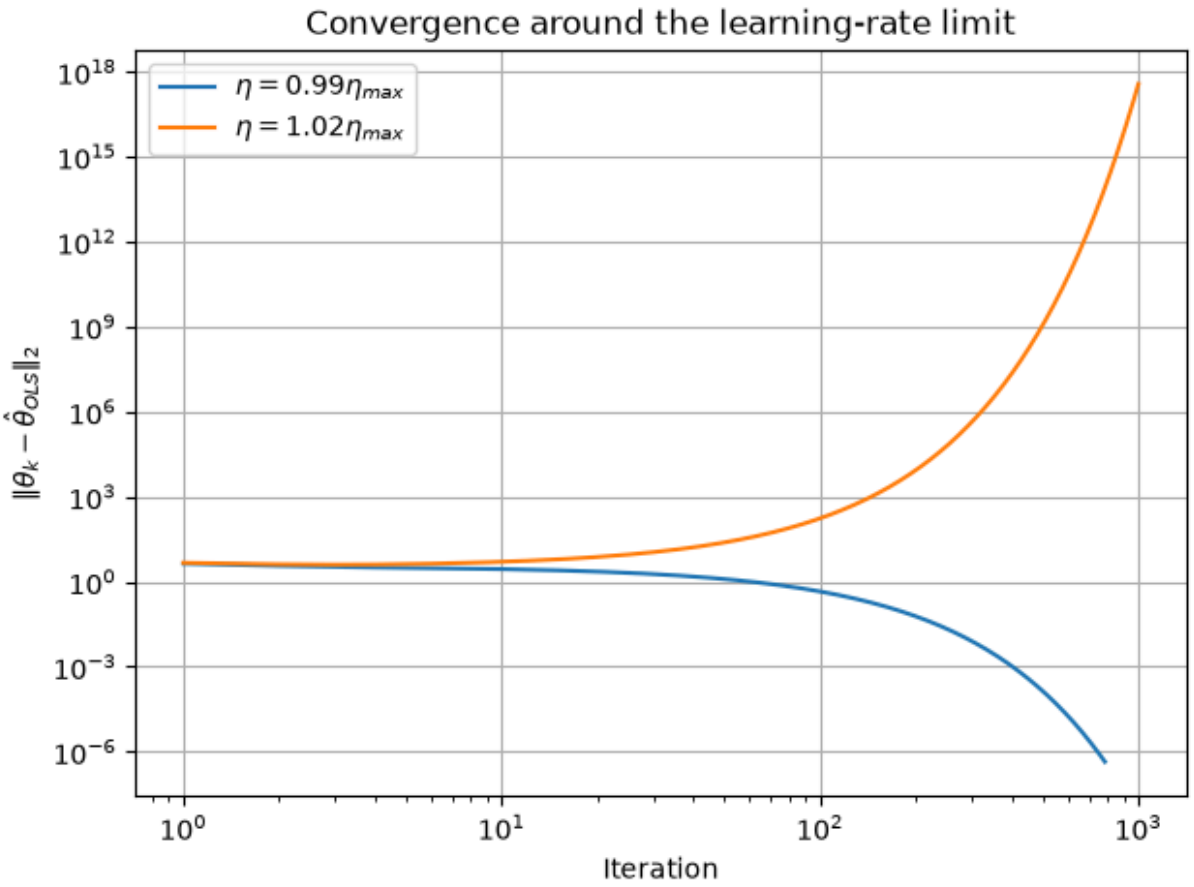
```
print(f"final distance: {dist_over[-1]:.4e}")


plt.figure(figsize=(7, 5))
plt.loglog(range(1, iter_under+1), dist_under, label=fr"$\eta = 0.99\eta_{{m
plt.loglog(range(1, iter_over+1), dist_over, label=fr"$\eta = 1.02\eta_{{max
plt.xlabel("Iteration")
plt.ylabel(r"$\|\theta_k-\hat{\theta}_{OLS}\|_2$")
plt.title("Convergence around the learning-rate limit")
plt.legend()
plt.grid()
plt.show()
```

```
Degree 2:
----------------------------
eta_max = 0.8941
eta_optimal = 0.5000
condition number = 1.2688
iterations: 9

eta below limit (0.99eta_max): 0.8851
final distance: 4.3767e-07

eta above limit (1.02eta_max): 0.9120
final distance: 3.6477e+17
```



Comment on degree-2 case:

We see from what the code prints that for a design matrix of degree 2, the maximal learning rate is $\eta_{\max} \approx 0.894$, while the optimal learning rate is $\eta_{\text{optimal}} \approx 0.500$, which agrees with the results from 4a) and b). The condition number is then $\kappa \approx 1.269$. Only 9 iterations was needed to converge.

We verify these results by using $\eta = 0.99\eta_{\max}$ and $\eta = 1.02\eta_{\max}$. This causes the algorithm to diverge, and we see from the plot that the absolute error (distance) between the exact solution (closed form) and the gradient descent solution is diverging for $\eta > 2/\lambda_{\max}$ (orange curve). For η smaller than the maximum value of η for convergence, we see that the solution converges to the closed-form solution (blue curve). That is, the theoretical bound $\eta < 2/\lambda_{\max}$ is correct.

In [11]:
```
# ------------------------------
# degree-5 design matrix case:
# ------------------------------

X5 = np.column_stack([x, x**2, x**3, x**4, x**5])
X5_norm, y_centered = standardize(X5, y)
H5 = (2 / len(y_centered)) * X5_norm.T @ X5_norm

eta_optimal5, eta_max5, kappa5 = compute_etas(H5)
theta_closed_formOLS5 = np.linalg.solve(X5_norm.T @ X5_norm, X5_norm.T @ y_c
theta5, iter5, dist5 = gradient_descent(X5_norm, y_centered, theta_closed_fo

# print results for degree-2 design matrix using OLS gradient descent
print("Degree 5:")
print("---------------------------------")
print(f"eta_max = {eta_max5:.4f}")
print(f"eta_optimal = {eta_optimal5:.4f}")
print(f"condition number = {kappa5:.4f}")
print(f"iterations: {iter5}\n")


# test Ridge instead of OLS using a lambda value that is not optimized
lmbda_test = 0.01
H2_Ridge = H2 + 2 * lmbda_test * np.eye(H2.shape[0])
H5_Ridge = H5 + 2 * lmbda_test * np.eye(H5.shape[0])

eta_opt_Ridge2, eta_max_Ridge2, kappa_Ridge2 = compute_etas(H2_Ridge)
eta_opt_Ridge5, eta_max_Ridge5, kappa_Ridge5 = compute_etas(H5_Ridge)

print("Testing with Ridge instead of OLS:")
print()
print(f"Degree 2 (lambda = {lmbda_test})")
print(f"{'Metric':<20}{'OLS':>12}{'Ridge':>12}")
print("-" * 44)
print(f"{'eta_max':<20}{eta_max:>12.4f}{eta_max_Ridge2:>12.4f}")
print(f"{'eta_optimal':<20}{eta_optimal:>12.4f}{eta_opt_Ridge2:>12.4f}")
print(f"{'condition number':<20}{kappa:>12.4f}{kappa_Ridge2:>12.4f}")
print()
print(f"Degree 5 (lambda = {lmbda_test})")
print(f"{'Metric':<20}{'OLS':>12}{'Ridge':>12}")
print("-" * 44)
```

```
print(f"{'eta_max':<20}{eta_max5:>12.4f}{eta_max_Ridge5:>12.4f}")
print(f"{'eta_optimal':<20}{eta_optimal5:>12.4f}{eta_opt_Ridge5:>12.4f}")
print(f"{'condition number':<20}{kappa5:>12.4f}{kappa_Ridge5:>12.4f}")
```

```
Degree 5:
---------------------------------
eta_max = 0.3538
eta_optimal = 0.3531
condition number = 520.3443
iterations: 3766

Testing with Ridge instead of OLS:

Degree 2 (lambda = 0.01)
Metric                        OLS       Ridge
-----------------------------------------------
eta_max                    0.8941      0.8862
eta_optimal                0.5000      0.4950
condition number           1.2688      1.2658

Degree 5 (lambda = 0.01)
Metric                        OLS       Ridge
-----------------------------------------------
eta_max                    0.3538      0.3526
eta_optimal                0.3531      0.3507
condition number         520.3443    183.8011
```

Comment on degree-5 case:

With a degree-5 design matrix, then $\eta_{\max} \approx 0.354$, while in the degree-2 case it was 0.894. The optimal learning rate is $\eta_{\text{optimal}} \approx 0.353$, which is very close to the maximum learning rate before the algorithm diverges. The condition number is now $\kappa \approx 520$, while the degree-2 case had 1.26, so $\kappa_5 \gg \kappa_2$. This is because the degree-5 design matrix has a wider range of eigenvalues, so $\lambda_{\min} \ll \lambda_{\max}$, making $\kappa = \lambda_{\max}/\lambda_{\min}$ much larger than in the degree-2 case. Now 3766 iterations are needed for convergence, in contrast to the 9 iterations nedded in the degree-2 case. This shows that adding higher-order terms in the design matrix makes the problem poorly conditioned (large spread in the eigenvalues of the Hessian matrix, resulting in large condtion number).

To check whether it is better to use Ridge regression, we tested this for both the degree-2 case and the degree-5 case. Ridge regression improves the condition number by a lot, because adding $2\lambda \boldsymbol{I}$ to the Hessian shifts the eigenvalues upwards. We tested by usig Ridge for a non-optimized value of λ. In teh dgeree-2 case the condition number did not improve much because the problem is already well-conditioned. However, in the degree-5 case, using Ridge reduces the condition number from 520 to 183, so using Ridge regression is better for stability in the 5-degree as the problem becomes more well-conditioned.

4d) Checking the gradient with automatic differentiation

Project 1, part e), asks you to compute the gradient in two independent ways: analytically, and by automatic differentiation. Write the OLS and Ridge cost functions as ordinary Python functions of $\boldsymbol{\theta}$ using `jax.numpy`, obtain their gradients with `jax.grad`, and compare with your analytical gradients from exercise 2 at a random $\boldsymbol{\theta}$ (Section 4.14.7 of the lecture notes; remember `jax.config.update("jax_enable_x64", True)`). Run your gradient descent code with the automatic gradient and check that you obtain the same iterates.

In one or two sentences: why is automatic differentiation neither symbolic nor numerical (finite-difference) differentiation, and what does a gradient cost relative to one evaluation of the cost function (Section 4.14.4)?

In [12]:
```
# ---------------------------------
#            Exercise 4d)
# ---------------------------------

import jax
import jax.numpy as jnp
from jax import grad

jax.config.update("jax_enable_x64", True)


def cost_OLS(theta, X, y):
    X_jax = jnp.array(X)
    y_jax = jnp.array(y)

    residual = X_jax @ theta - y_jax
    return jnp.mean(residual**2)


def cost_Ridge(theta, X, y, lmbda):
    X_jax = jnp.array(X)
    y_jax = jnp.array(y)

    residual = X_jax @ theta - y_jax
    return jnp.mean(residual**2) + lmbda * jnp.sum(theta**2)


def gradient_OLS_ad(theta):
    return grad(cost_OLS)(theta, jnp.asarray(X_norm), jnp.asarray(y_centered


def gradient_Ridge_ad(theta):
    return grad(cost_Ridge)(theta, jnp.asarray(X_norm), jnp.asarray(y_center


def gradient_descent_AD(gradient_function, X, eta, num_iters=1000, tol=1e-8)
    """
    Performs gradient descent using gardients computed with automatic
    differentiation using JAX.
    """
    theta = jnp.zeros(X.shape[1])
```

```
    for i in range(num_iters):
        gradient = gradient_function(theta)

        theta -= eta * gradient

        if jnp.linalg.norm(gradient) < tol:
            break

    return np.array(theta), i + 1

# generate random theta
theta_test = rng.standard_normal(n_features)
theta_test_jax = jnp.array(theta_test)

# evaluate ad gradients with test theta
grad_OLS_ad_test = gradient_OLS_ad(theta_test_jax)
grad_Ridge_ad_test = gradient_Ridge_ad(theta_test_jax)

# evaluate analytical gradients with test theta
grad_OLS_analytical = (2/n) * X_norm.T @ (X_norm @ theta_test - y_centered)
grad_Ridge_analytical = (2/n) * X_norm.T @ (X_norm @ theta_test - y_centered

# perform gradient descent with ad gradients
theta_OLS_ad, iters_OLS_ad = gradient_descent_AD(gradient_OLS_ad, X_norm, et
theta_Ridge_ad, iters_Ridge_ad = gradient_descent_AD(gradient_Ridge_ad, X_no

# perform gradient descent with analytical gradients
theta_OLS_analytic, iters_OLS_analytic, _ = gradient_descent(X_norm, y_cente
theta_Ridge_analytic, iters_Ridge_analytic, _ = gradient_descent(X_norm, y_c


# print ad and analytical gradients for a random theta
print("OLS:")
print("-------------------------------------")
print(f"Analytical gradient: {grad_OLS_analytical}")
print(f"AD gradient:         {grad_OLS_ad_test}")
print("max |AD - analytical| =", np.max(np.abs(np.asarray(grad_OLS_ad_test)
print()
print("Ridge:")
print("-------------------------------------")
print(f"Analytical gradient: {grad_Ridge_analytical}")
print(f"AD gradient:         {grad_Ridge_ad_test}")
print("max |AD - analytical| =", np.max(np.abs(np.asarray(grad_Ridge_ad_test

# print gradient descent comparison
print("\nGradient descent comparison:")
print("-------------------------------------")
print("OLS:")
print("AD:         ", theta_OLS_ad)
print("Analytical: ", theta_OLS_analytic)
print("Difference:", np.linalg.norm(theta_OLS_ad - theta_OLS_analytic))
print(f"iterations ad = {iters_OLS_ad}")
print(f"iterations analytical = {iters_OLS_analytic}")

print("\nRidge:")
```

```
print("AD:         ", theta_Ridge_ad)
print("Analytical: ", theta_Ridge_analytic)
print("Difference:", np.linalg.norm(theta_Ridge_ad - theta_Ridge_analytic))
print(f"iterations ad = {iters_Ridge_ad}")
print(f"iterations analytical = {iters_Ridge_analytic}")
```

```
OLS:
-------------------------------------
Analytical gradient: [  1.3472716  -10.98334769]
AD gradient:         [  1.3472716  -10.98334769]
max |AD - analytical| = 4.440892098500626e-16

Ridge:
-------------------------------------
Analytical gradient: [  1.35041384 -10.98093569]
AD gradient:         [  1.35041384 -10.98093569]
max |AD - analytical| = 4.440892098500626e-16

Gradient descent comparison:
-------------------------------------
OLS:
AD:          [-1.0773505  5.854734 ]
Analytical:  [-1.07735048  5.854734  ]
Difference: 2.6971212819688254e-08
iterations ad = 11
iterations analytical = 9

Ridge:
AD:          [-1.06681834  5.81907777]
Analytical:  [-1.06681832  5.81907778]
Difference: 2.7377794632738766e-08
iterations ad = 11
iterations analytical = 9
```

Comment 4d):

We see from what the code prints that automatic differentiation gradients agree with the analytical gradients for both OLS and Ridge to machine precision. The gradient descent using automatic differentiation agrees with the gardient descent using the analytical gradient, with maximum error of order 10^{-8} for both OLS and Ridge. Thus, the automatic differentiation gives the same results as analytic gradient descent. We see that using automatic differentiation we need two more iterations than analytic gradient descent to converge in both cases.

Automatic differentiation is different from finite-difference differentiation and symbolic differentiation because it does not approximate the derivative or manipulate a symbolic expression. AD computes the derivatives by applying the chain rule to the numerical operations of the cost function, while finite difference differentiation approximate the derivative using small perturbations and evaluation of a function at different points. This is prone to numerical round-off errors. Using reverse-mode AD to evaluate a gradient costs about 2-3 evaluations of the cost function (section 4.14).

Exercise 5: Ridge regression and a new synthetic data set

We create a synthetic linear regression data set with a sparse underlying relationship: many features, of which only a few contribute to the target. We use 10 features with only 3 non-zero weights in the true model, so that the target is a linear combination of a few features (with known coefficients) plus random noise. The steps are:

- decide on the number of samples and features (100 samples, 10 features);
- define the *true* coefficient vector with mostly zeros, for example $\boldsymbol{\theta}_{\text{true}} = [5.0, -3.0, 0, 0, 0, 0, 2.0, 0, 0, 0]$, meaning only features 0, 1 and 6 have a real effect on y;
- sample the feature matrix $\boldsymbol{X}$ from a standard normal distribution, so that the features are roughly centred around 0;
- compute $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\theta}_{\text{true}} + \boldsymbol{\varepsilon}$ with Gaussian noise $\boldsymbol{\varepsilon}$.

In [13]:
```
import numpy as np

# Set random seed for reproducibility
np.random.seed(0)

# Define dataset size
n_samples = 100
n_features = 10

# Define true coefficients (sparse linear relationship)
theta_true = np.array([5.0, -3.0, 0.0, 0.0, 0.0, 0.0, 2.0, 0.0, 0.0, 0.0])

# Generate feature matrix X (n_samples x n_features) with random values
X = np.random.randn(n_samples, n_features)  # standard normal distribution

# Generate target values y with a linear combination of X and theta_true, pl
noise = 0.5 * np.random.randn(n_samples)    # Gaussian noise
y = X @ theta_true + noise
```

This code produces a data set where only features 0, 1 and 6 significantly influence $\boldsymbol{y}$; the rest have zero true coefficient. Feature 0 has a true weight of 5.0, feature 1 of -3.0 and feature 6 of 2.0, so the expected relationship is

$$y \approx 5x_0 - 3x_1 + 2x_6 + \text{noise}.$$

You can remove the noise if you wish.

Fit the above data set using OLS and Ridge regression with the analytical expressions and with your own gradient descent code (scale the data as in exercise 1). If everything works, the learned coefficients should be close to the true values $[5.0, -3.0, 0, \ldots, 2.0, \ldots]$. Due to regularisation and noise the learned values will not exactly equal the true ones, but they should be in the same ballpark. Which method, OLS or Ridge, gives the best results — and

best in which sense? Compute the condition number of the Hessian for this data set: why does gradient descent converge so much more easily here than for the degree-5 polynomial of exercise 4c)?

Exercise 5 solution

In [14]:
```
x_norm, y_centered = standardize(X, y)


lmbda_ridge = 0.01
theta_OLS, theta_ridge  = closed_form_thetas(x_norm, y_centered, lmbda = lmb

print(f"True coefficients:\n {theta_true}")
print(f"OSL coefficients:\n {theta_OLS}\n")
print(f"Ridge coefficients (lambda = {lmbda_ridge}:\n {theta_ridge})\n")


theta_gd_ols, iter_ols, _ = gradient_descent(x_norm, y_centered, theta_OLS,
theta_gd_ridge, iter_ridge, _ = gradient_descent(x_norm, y_centered, theta_r


H = (2 / n_samples) * x_norm.T @ x_norm
eta_optimal, eta_max, kappa = compute_etas(H)
print(f"Condition number: {kappa}")
print(f"Gradient Desent OLS: {iter_ols} iterations")
print(f"Gradient Desent Ridge: {iter_ridge} iterations")
```

```
True coefficients:
 [ 5. -3.  0.  0.  0.  0.  2.  0.  0.  0.]
OSL coefficients:
 [ 5.03241281e+00 -2.89258175e+00 -1.55189951e-02  1.51795012e-01
 -6.83299260e-02 -4.40147965e-02  1.76999871e+00  4.37643569e-03
  4.52550260e-02 -4.97610000e-02]

Ridge coefficients (lambda = 0.01:
 [ 4.98041921 -2.85683951 -0.01549982  0.15658961 -0.07810233 -0.05094777
  1.75377987  0.00679113  0.0343071  -0.04883458])

Condition number: 2.266016508156028
Gradient Desent OLS: 114 iterations
Gradient Desent Ridge: 112 iterations
```

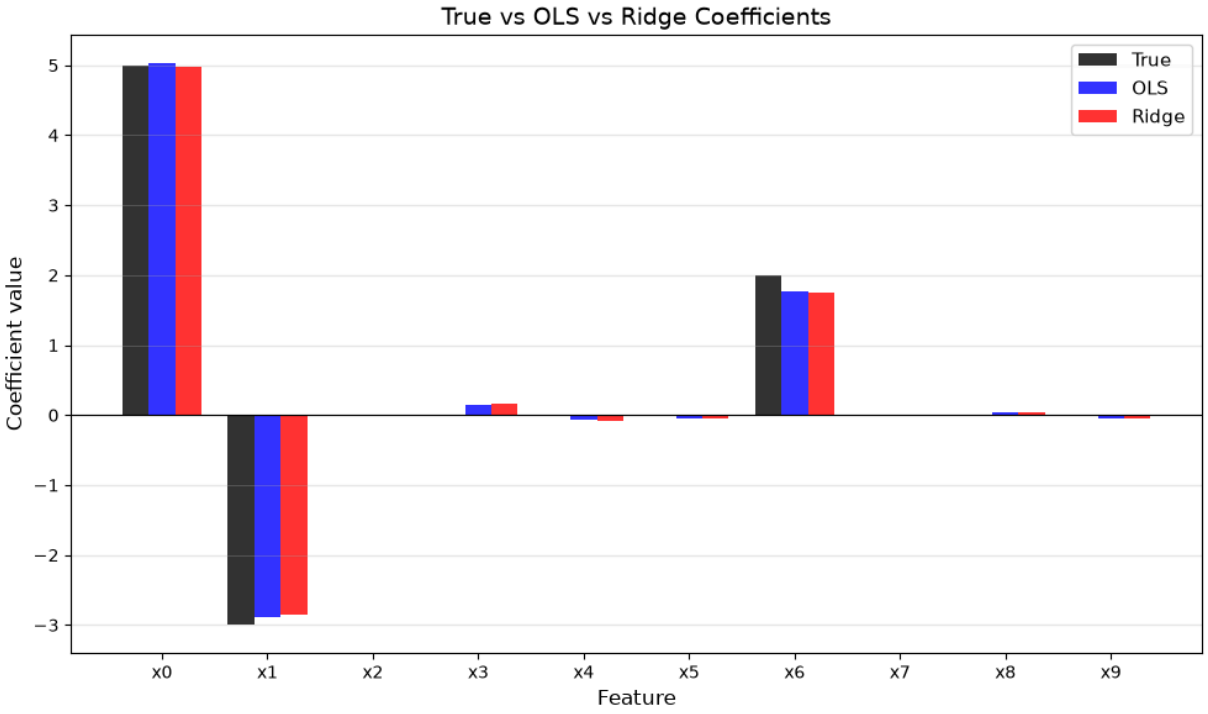
In [15]:
```
fig, ax = plt.subplots(figsize=(10, 6))

x = np.arange(n_features)
width = 0.25


bars1 = ax.bar(x - width, theta_true, width, label='True', alpha=0.8, color=
bars2 = ax.bar(x, theta_OLS, width, label='OLS', alpha=0.8, color='blue')
bars3 = ax.bar(x + width, theta_ridge, width, label='Ridge', alpha=0.8, colo
```

```
ax.set_xlabel('Feature', fontsize=12)
ax.set_ylabel('Coefficient value', fontsize=12)
ax.set_title('True vs OLS vs Ridge Coefficients', fontsize=14)
ax.set_xticks(x)
ax.set_xticklabels([f'x{i}' for i in range(n_features)])
ax.legend(fontsize=11)
ax.axhline(y=0, color='k', linestyle='-', linewidth=0.8)
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```



For the spare synthetic data, OLS and Ridge preform similarly. Both successfully recover the true non-zero coefficeints and produce small moise amoung the true zeroes. With the chosen λ, Ridge only shrinks coefficeients slightly. Thus the Ridge regression would proove more beneficial amoung highly correlated features or more aggresive regularization. For indepentandt random features, OLS is already well-conditioned.

The condion muber for the sparse data is $\kappa \approx 2.27$, compreare to $\kappa \approx 520$ for the degree-5 polynomial. This mich smaller κ explain why gradient decent converges so much faster on sparese random data. The random independant features produce a well-conditioned Hessian with balanced eigenvalue, apposed to the polynomial who has highly correlated features and produce a problem with large eigenvalue spread.

Exercise 6: adding momentum

Part f) of Project 1 asks you to extend your gradient descent code with momentum and, next week, with the adaptive methods AdaGrad, RMSProp and Adam. Start here with

momentum, Eq. (4.23) of the lecture notes,

$$\boldsymbol{v}_{k+1} = \gamma \boldsymbol{v}_k + \eta \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}_k), \qquad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \boldsymbol{v}_{k+1},$$

which is one line more than the code of exercise 4. Use the degree-5 polynomial data of exercise 4c), where plain gradient descent is slow, and the learning rate $\eta = 0.9 \cdot 2/\lambda_{\max}$.

1. Run $\gamma \in \{0, 0.5, 0.9\}$ and count the iterations needed to reach $\|\boldsymbol{\theta}_k - \hat{\boldsymbol{\theta}}\|_2 < 10^{-6}$. Plot the three convergence curves on a logarithmic scale.
2. Section 4.6 shows that, with optimally tuned parameters, momentum reduces the iteration count from $\propto \kappa$ to $\propto \sqrt{\kappa}$, Eq. (4.26). How large a reduction do you observe with the fixed η above, and how does it compare with $\sqrt{\kappa}$? Try the optimal momentum $\gamma^* = \left((\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)\right)^2$.
3. Explain, in terms of Eqs. (4.24) and (4.25), why momentum speeds up the flat directions and damps the oscillations in the steep ones.
4. (Optional) Repeat with $\eta = 1.5 \cdot 2/\lambda_{\max}$, where plain gradient descent diverges. What happens with momentum, and why?

Exercise 6 solution

In [16]:
```
rng = np.random.default_rng(2026)
n = 100
x = rng.uniform(-2.0, 2.0, n)
y = 2.0 - x + 5.0 * x**2 + 0.5 * rng.standard_normal(n)

gamma_list = np.array([0, 0.5, 0.9])
X5 = np.column_stack([x, x**2, x**3, x**4, x**5])
X5_norm, y_centered = standardize(X5, y)
H5 = (2 / len(y_centered)) * X5_norm.T @ X5_norm
theta_closed_formOLS5, _ = closed_form_thetas(X5_norm, y_centered, lmbda = 0

lmbda_max5 = np.linalg.eigvalsh(H5).max()
eta5 = 0.9 * 2 / lmbda_max5

results = {}



def gradient_descent_extended(X, y, theta_exact, eta, gamma, method="OLS", l
    """
    Performs gradient descent using num_iters number of iterations using OLS
    Stops when the gradient converges for a chosen tolerance.
    (theta, iterations, distances) is returned.
    """
    n = len(y)
    theta = np.zeros(X.shape[1])   # starting guess is zero
    v = np.zeros(X.shape[1])

    distances = []
```

```
    for i in range(num_iters):
        if method == "OLS":
            gradient = (2/n) * X.T @ (X @ theta - y)

        elif method == "Ridge":
            gradient = (2/n) * X.T @ (X @ theta - y) + 2 * lmbda * theta

        else:
            raise RuntimeError("Please choose regression method: OLS or Ridg

        # update velocity
        v = gamma * v + eta * gradient

        # update theta
        theta -= v

        # compare to closed-form solution
        distance = np.linalg.norm(theta - theta_exact)
        distances.append(distance)

        # stopping criterion if gradient converges
        if distance < tol:
            break

    return theta, i + 1, np.array(distances)

for gamma in gamma_list:
    theta, iterations, distances = gradient_descent_extended(X5_norm, y_cent

    results[gamma] = distances
    print(f"gamma = {gamma}: {iterations} iterations")
```

```
gamma = 0.0: 3689 iterations
gamma = 0.5: 1829 iterations
gamma = 0.9: 284 iterations
```

In [17]:

```
_, _, kappa5 = compute_etas(H5)
iterations_list = [3689, 1829, 284]
gamma_opt = ((np.sqrt(kappa5) - 1) / (np.sqrt(kappa5) + 1))**2

print(f"Condition number = {kappa5:.4f}")
print(f"sqrt(kappa) = {np.sqrt(kappa5):.4f}")
print(f"Optimal gamma = {gamma_opt:.4f}")

theta, iterations, distances = gradient_descent_extended(X5_norm, y_centered

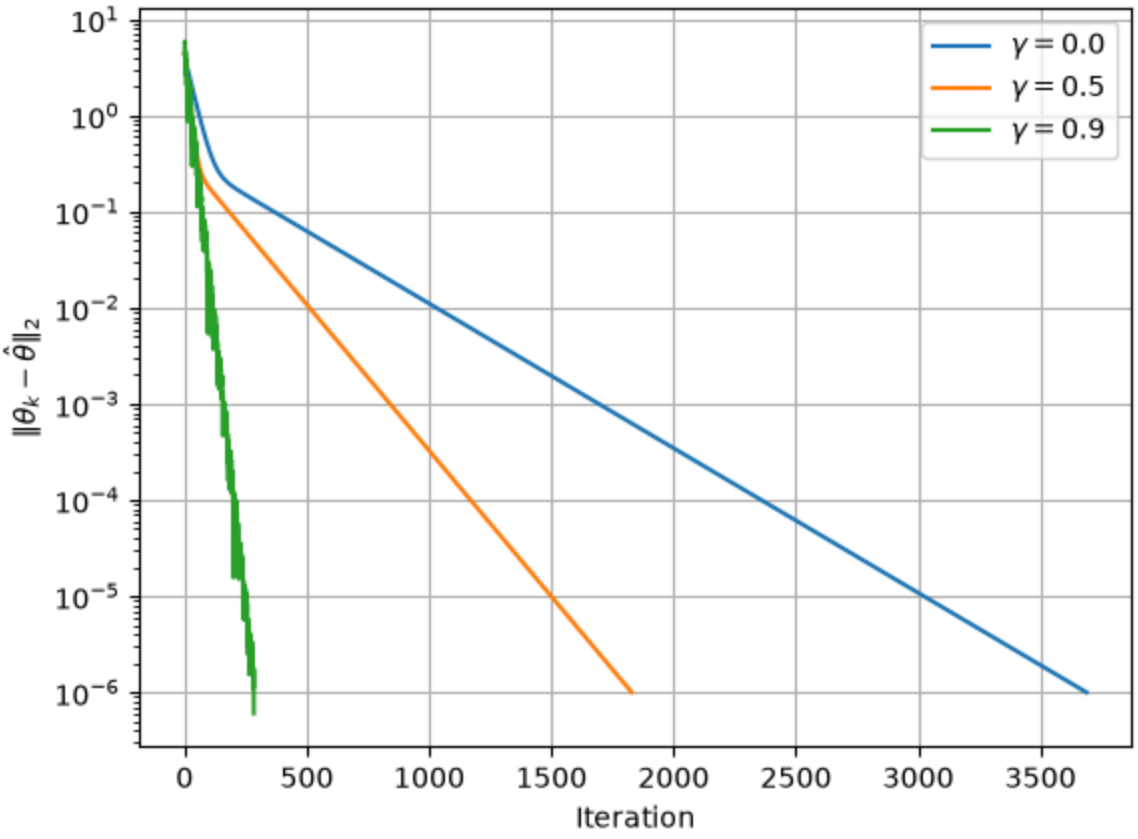
print(f"Iterations with optimal gamma = {iterations}")
print(f"Reduction gamma = 0.9: {iterations_list[0]/iterations_list[2]}")
print(f"Reduction optimal gamma: {iterations_list[0]/iterations}")
```

```
Condition number = 520.3443
sqrt(kappa) = 22.8111
Optimal gamma = 0.8391
Iterations with optimal gamma = 517
Reduction gamma = 0.9: 12.98943661971831
Reduction optimal gamma: 7.1353965183752415
```

In [18]:
```
plt.figure()

for gamma in gamma_list:
    plt.semilogy(
        results[gamma],
        label=f"$\\gamma={gamma}$"
    )

plt.xlabel("Iteration")
plt.ylabel(r"$\|\theta_k-\hat{\theta}\|_2$")
plt.legend()
plt.grid()
plt.show()
```



When comprearing with $\gamma = 0.9$ we get a reduction of 12.9 and for optimal γ a reduction of 7.1 which is quite far off $\sqrt(\kappa) = 22.8$. This is likely due to η not beeing optimized along side γ, this leads to the γs giving great reduction, but not close to the optimal.

When in flat reagions the gradiant reamins almost constant thus the velocity accumilates $v_\infty = \eta g \sum_j \gamma^j = \frac{\eta g}{1-\gamma}$ (4.25) for $\gamma = 0.9$ one gets $10\eta g$ which is ten times faster than regualr gradient desent.

When in steep areas you overshoot the minimum and this the gradient change sign, when looking at $v_{k+1} = \eta \sum_{j=0}^{k} \gamma^{k-j} \nabla C(\theta_j)$ (4.24) every other term is negative and positiv, hence slowing down the momentum insuring you can reliably hit the minimum.

Insights you should have gained this week — a self-check

Before you hand in, go through the list below without looking at the notes. Each row is a question we may ask in the Tuesday session or on Project 1; the right-hand column is the insight it tests, in one sentence.

Ask yourself	The insight it tests
My loop stopped after its allowed iterations with a small but nonzero gradient. Has it converged?	No. Along the slowest direction the error contracts by $\lvert 1 - \eta\lambda_{\min}\rvert$ per step, Eq. (4.19), and can still be $\lVert\nabla C\rVert/\lambda_{\min}$ large. Stop on the gradient (or on $\Delta\boldsymbol{\theta}$), never on a count alone.
Why does gradient descent care about the scaling of the columns when the closed form does not?	The closed form solves the normal equations in one go; gradient descent takes the same step in every direction, and the column scales set the eigenvalue spread $\kappa(\boldsymbol{H}) = \kappa_2(\boldsymbol{X})^2$, Eq. (4.22), hence the iteration count. Same minimum, different road.
Ridge converged in far fewer iterations than OLS. Did it find a better OLS solution?	No — it found the exact solution of a *different* problem, Eq. (4.16). The penalty lifts $\lambda_{\min}$, Eq. (4.17), which shortens the road, but the destination moved: bias traded for variance.
The same code diverged at $\eta = 0.5$ on one data set and converged at $\eta = 0.8$ on another. Why?	Stability requires $\eta < 2/\lambda_{\max}$, Eq. (4.20), and $\lambda_{\max}$ is a property of $\boldsymbol{X}$, not of the algorithm. Compute it — `np.linalg.eigvalsh(2/n * X.T @ X).max()` — before you choose η.
The closed form and my converged gradient descent disagree in the second decimal for Ridge. Which is wrong?	Possibly neither: with the cost $\frac{1}{n}\lVert\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\rVert^2 + \lambda\lVert\boldsymbol{\theta}\rVert^2$ the closed form has $n\lambda\boldsymbol{I}$, Eq. (3.95), and `scikit-learn`'s `alpha` is $n\lambda$. Compare cost functions before comparing methods.
My JAX gradient agrees with my analytical gradient to 10^{-15}. What have I proved?	That your derivation matches the cost you *wrote down* — an algebra error would show as 10^{-2}. Not that the cost is the one you meant (a missing $1/n$, a penalised intercept): both gradients would then agree and both would be wrong.
Momentum converged where plain gradient descent diverged. Is the learning rate now unimportant?	No. The stability edge moves to $2(1+\gamma)/\lambda_{\max}$, and the best γ is tied to κ through Eq. (4.26); too much momentum rings for ever. Two data-dependent knobs instead of one.

Ask yourself	The insight it tests
Why does momentum help, in one sentence?	It adds up gradients that keep pointing the same way (the flat directions, effective step $\eta/(1-\gamma)$, Eq. (4.25)) and cancels gradients that alternate in sign (the steep direction), so the iteration count scales with $\sqrt{\kappa}$ instead of κ.
I swap the OLS cost for a neural-network cost and keep `jax.grad`. What changes?	In the code: only the definition of the cost. In the theory: convexity is gone, so a stationary point need not be the global minimum (Section 4.1) and the bound of Eq. (4.20) is only local — good behaviour becomes empirical, not a theorem.
Why reverse mode for training, and what is its price?	One scalar output and p inputs: reverse mode returns the full gradient for two or three cost evaluations, whatever p (the cheap gradient principle, Section 4.14.4); forward mode or finite differences would need p sweeps. The price is the tape — every intermediate value is stored until the backward sweep has used it: memory, not time.
Why not just use finite differences to check the gradient, and be done with it?	For a *quadratic* cost the central difference is exact up to rounding, so it would pass; for anything else its best accuracy is about 10^{-11} at $h \approx 10^{-6}$, Eq. (4.46), and it costs $2p$ evaluations. Use it as a check at $h \approx 10^{-5}$, not as the gradient.

Practicalities

Deliver your solutions (a single Jupyter notebook, with the analytical parts either typeset in Markdown/LaTeX or as a scanned handwritten note) in Canvas before **Friday September 11 at midnight**. Collaboration is encouraged — list your collaborators.